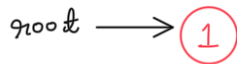


## Properties of Binary Trees

For the following properties, let us assume that the height of the tree is  $h$ . Also, assume that root node is at height Zero.

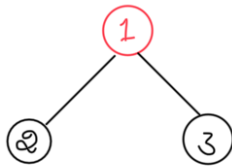


Height

$$h = 0$$

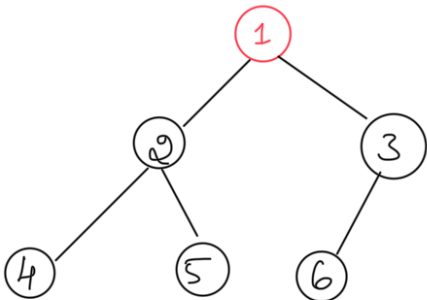
Number of nodes at level  $h$ 

$$2^0 = 1$$



$$h = 1$$

$$2^1 = 2$$



$$h = 2$$

$$2^2 = 4$$

From the diagram we can infer the following properties

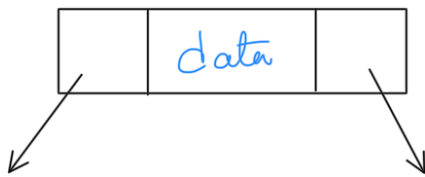
- The number of nodes  $n$  in a full Binary tree is  $2^{h+1} - 1$ . Since, there are  $h$  levels we need to add all nodes at each level  $[2^0 + 2^1 + 2^2 + \dots + 2^h]$   
 $\Rightarrow 2^{h+1} - 1 = 2^0 + 2^1 + 2^2 + \dots + 2^h$
- The number of nodes  $n$  in a complete Binary tree is between  $2^h$  (minimum) and  $2^{h+1} - 1$  (maximum). For more information on this, refer to priority Queues chapter.

- The number of leaf nodes in a full Binary tree is  $2^h$ .
- The number of Null links (wasted pointers) in a Complete Binary tree of  $n$  nodes is  $n+1$ .

## Structure of Binary Trees

Now let us define class of a Binary Tree. For simplicity assume that the data of the nodes are integers.

One way to represent a node (which contains data) is to have two links which point to left and right children along with data fields as shown below.



OR



Class BinaryTreeNode {

int data;

BinaryTreeNode left;

BinaryTreeNode right;

}

NOTE: In Trees, the default flow is from parent to child and it is not mandatory to show directed branches.



# Operations on Binary Trees

## Basic Operations

Inserting an Element Into a tree  
Deleting an Element Into a tree  
Searching for an element  
Traversing the tree

## Auxiliary Operations

Finding the Size of the tree  
Finding the height of the tree  
Finding the level which has Maximum Sum  
Finding the least common ancestor (LCA) for a given pair of nodes, and many more

## Applications of Binary Trees

Following are the Some of the applications where Binary trees play an Important Role

- Expression trees are used in compilers
- Huffman coding trees are used in data compression Algorithms
- Binary Search Trees (BST), which supports Search, Insertion and deletion on a collection of items in  $O(\log n)$
- Priority Queue (PQ), which supports Search and deletion of minimum or Maximum on a collection of items in logarithmic time (in worst case)